

정현파(sin/cos) 발생용 디지털 필터

- 선형인 이산적 시스템에 대해서 그 입력 신호의 z 변환을 $X(z)$, 출력 신호의 z 변환을 $Y(z)$, 이산적 시스템의 전달 함수를 $H(z)$ 라고 하면, $X(z)$ 와 $Y(z)$ 와의 관계는 다음과 같이 나타낼 수 있다.

$$Y(z) = H(z) \cdot X(z)$$

- 따라서 전달 함수 $H(z)$ 가 $\sin$ 의 z 변환이 되는 시스템을 작성하고 이 시스템에 $X(z) = 1$ 인 신호를 입력하면, 출력 신호의 z 변환은 $\sin$ 의 z 변환과 같아지게 된다.
- $X(z) = 1$ 인 신호를 $x[n]$ 이라고 하면, 그것은 단위 임펄스 신호 $\delta[n]$ 이고, 다음과 같다.

$$x[n] = \delta[n] = \begin{cases} 1, n = 0 \\ 0, n \neq 0 \end{cases}$$

- 한편 각주파수 $\Omega_o (= 2\pi F_o)$ 의 사인파를 표본화 간격 T 로 표본화한 이산적 신호를 $\sin [n\Omega_o T]$ 로 한다.
- 이 이산적 사인파의 z 변환은 다음과 같다

$$\frac{(\sin \Omega_o T) z^{-1}}{1 - 2 (\cos \Omega_o T) z^{-1} + z^{-2}}$$

- Cos의 경우는 다음과 같다.

$$\frac{1 - (\cos \Omega_o T) z^{-1}}{1 - 2 (\cos \Omega_o T) z^{-1} + z^{-2}}$$

- 따라서 사인파 발생 시스템의 입출력 관계는 다음과 같이 나타낼 수 있다.

$$Y(z) = \frac{(\sin \omega_o T) z^{-1}}{1 - 2 (\cos \omega_o T) z^{-1} + z^{-2}} \cdot X(z)$$

이 식은 다음과 같이 바꿔 쓸수 있다.

$$Y(z) - 2 (\cos \omega_o T) Y(z)z^{-1} + Y(z)z^{-2} = (\sin \omega_o T) X(z)z^{-1}$$

- 이 식에 나타나는 z 변환된 신호에 대해서는 z 변환의 성질 중 "시간축 상의 시프트"를 적용하면 시간 영역에서의 표현과 그 z 변환의 관계는 다음과 같이 정리할 수 있다.

시간영역의 표현	z 영역의 표현(z 변환)
$x[n]$	$X(z)$
$x[n-1]$	$X(z)z^{-1}$
$y[n]$	$Y(z)$
$y[n-1]$	$Y(z)z^{-1}$
$y[n-2]$	$Y(z)z^{-2}$

- 이 관계를 고려하면 다음과 같은 시간 영역 표현으로 변환할 수 있다.

$$y[n] = 2 (\cos \Omega_o T) y[n-1] + y[n-2] + (\sin \Omega_o T) x[n-1]$$

- 이것은 IIR 필터의 일종으로, 입력 신호 $x[n]$ 과 출력 신호 $y[n]$ 의 관계는 다음과 같은 차분방정식으로 나타낼 수 있다.

$$y[n] = a_1 y[n-1] + y[n-2] + b_1 x[n-1]$$

$$a_1 = 2 \cos(\Omega_0 T) = 2 \cos(2\pi F_0 T)$$

$$b_1 = \sin(\Omega_0 T) = \sin(2\pi F_0 T)$$

- 여기서 F_0 는 발생하는 사인파의 주파수, T 는 샘플(표본화) 간격을 의미한다.
- 이 식에서 a_l 은 발생한 사인파의 주파수를 결정하는 계수이고, b_1 은 발생한 사인파의 진폭을 결정하는 계수이다.
- 앞에서 얻은 최종 차분 방정식을 C언어로 변환한다면 다음과 같은 코드로 구현할 수 있다.

```

/*****
/* Sin Generator using IIR filter      */
/* frequency of sin : 880 Hz          */
*****/
#include <math.h>

const float PI2 = 6.283185307;
const float T0 = 1.0/8000.0;
const float F0 = 880.0;

void main(){
    float a1, b0, yn0, yn1, yn2;
    short tmp;

    Init();          // Initialize H/W

    a1=2.0*cos (PI2*F0*T0);    // 880 Hz
    b0 = 16384.0 *sin(PI2*F0*T0);

    yn2 = 0.0;        /* Initial value : y[0] = 0 */
    yn1 = b0;         /* Initial value : y[1] = b0 */

    while (1){
        ReadRdy();    /* synchronize */

        yn0 = a1*yn1 - yn2; /* y[n] = a1*y[n-1]-y[n-2] */
        yn2 = yn1;         /* y[n-2] = y[n-1] */
        yn1 = yn0;         /* y[n-1] = y[n] */
        tmp = (short)yn0 & 0xFFFE;
        BSPOWrite(tmp);    /* Output */
    }
}

```